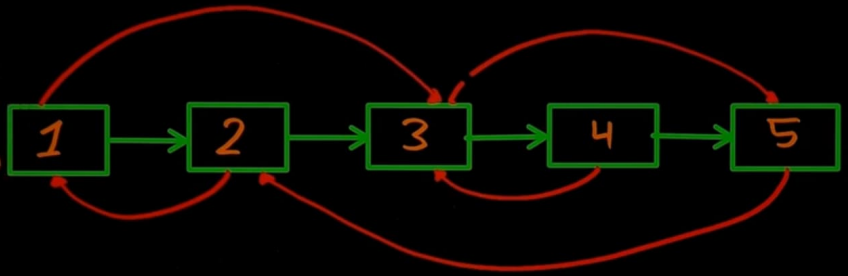
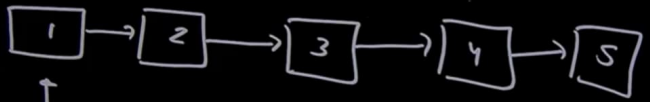
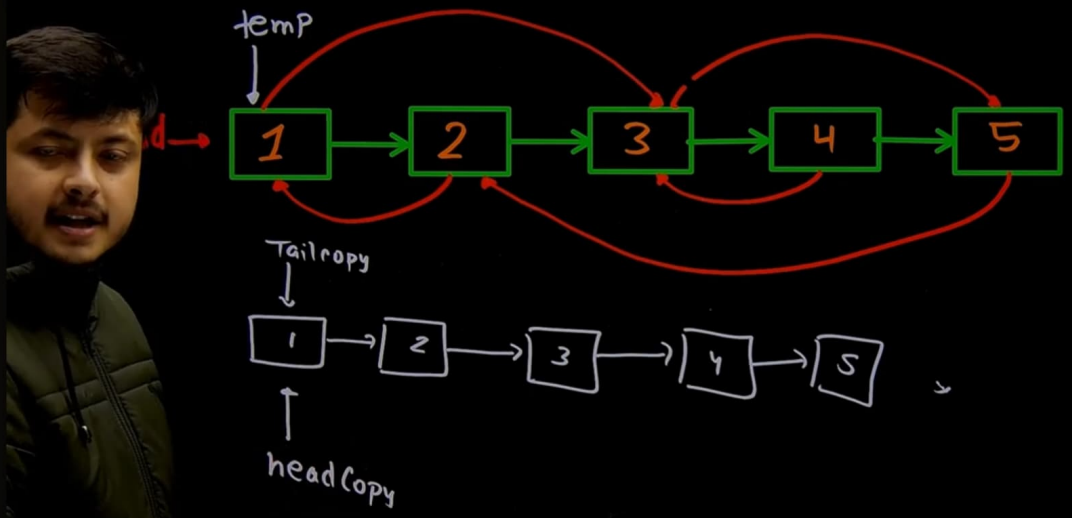
Clone a Linked List




```
Class Node
{ public:
  int data;
  Node *next,
       *arb
  };
```

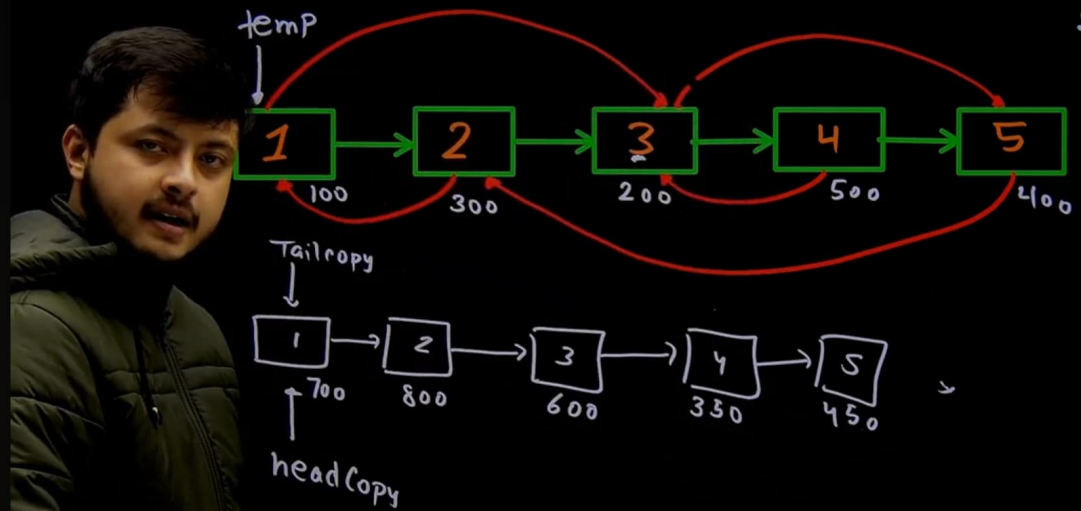


Clone a Linked List



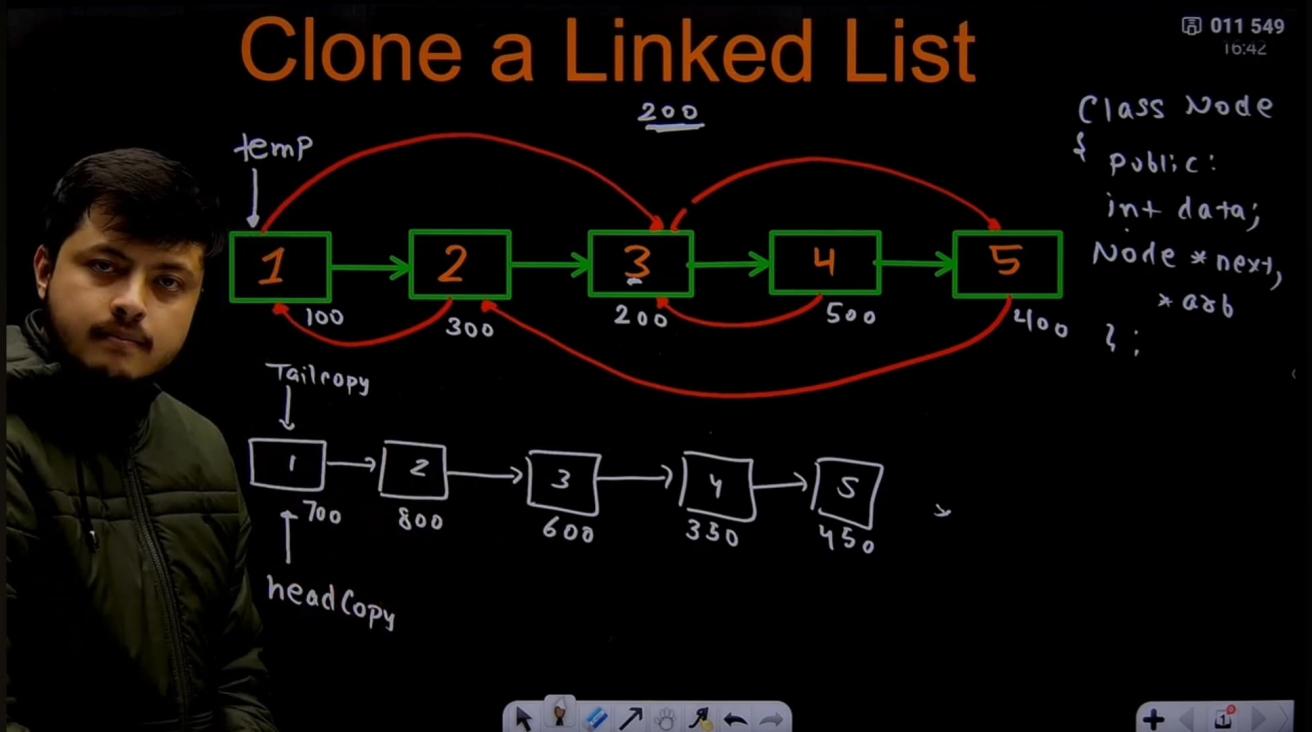

```
Class Node
{ public:
  int data;
  Node *next,
       *arb
};
```

Clone a Linked List

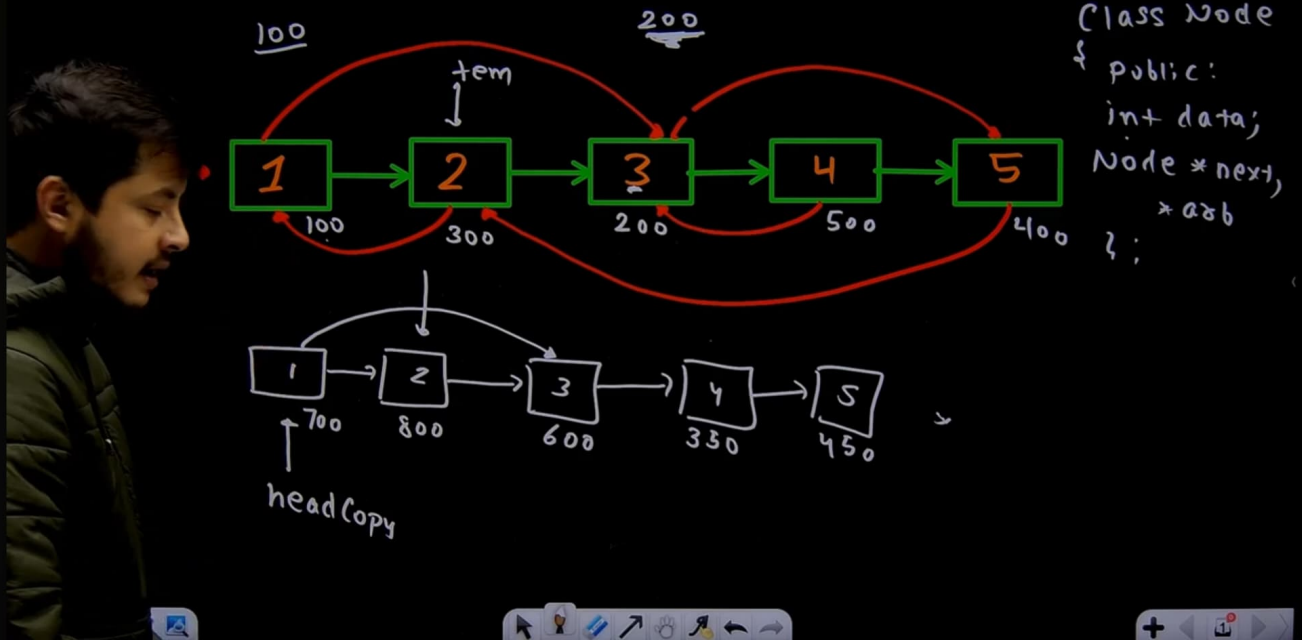



```
Class Node
{ public:
  int data;
  Node *next,
      *arb
};
```

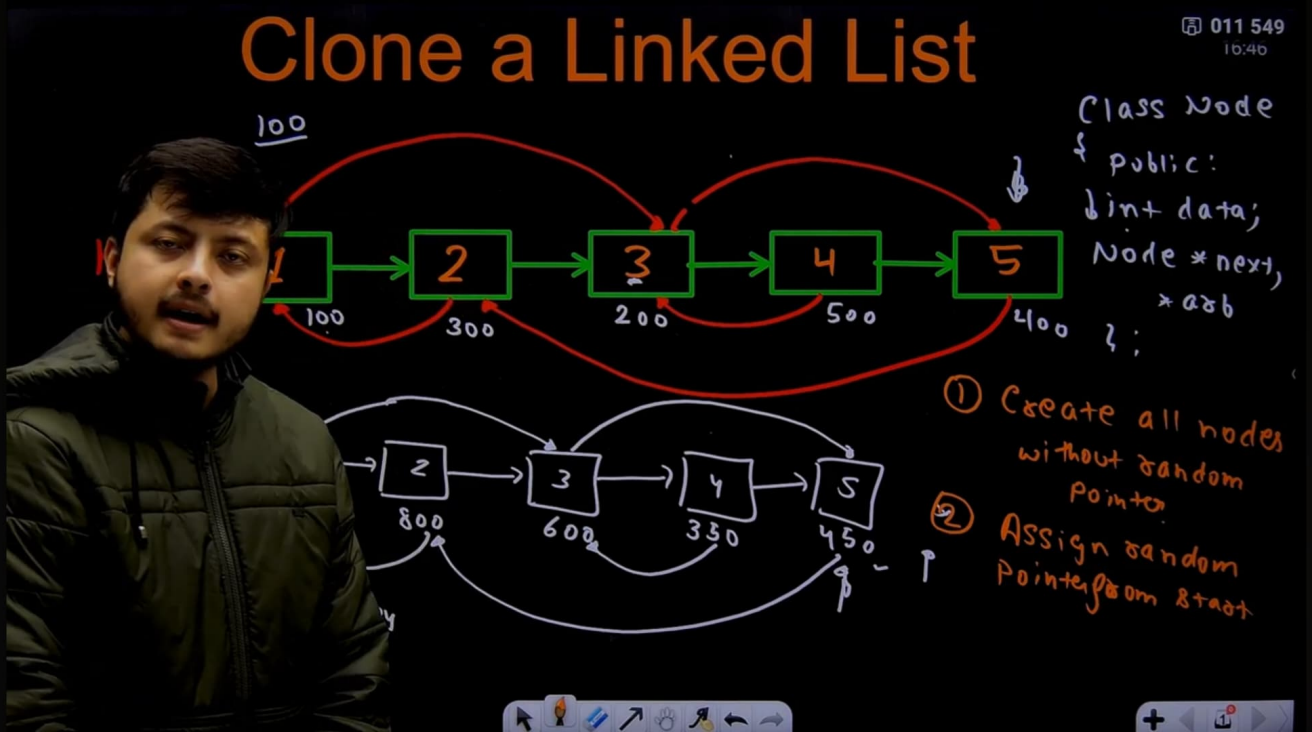
Clone a Linked List



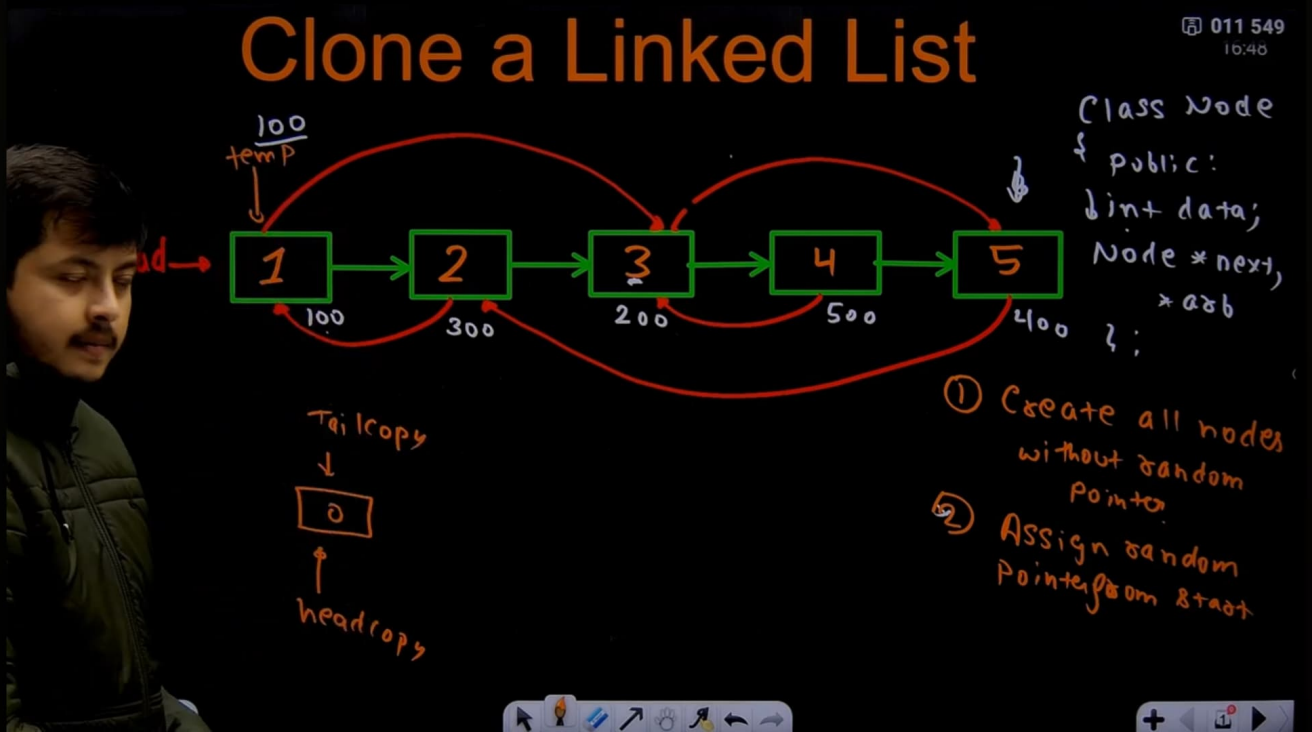
Clone a Linked List

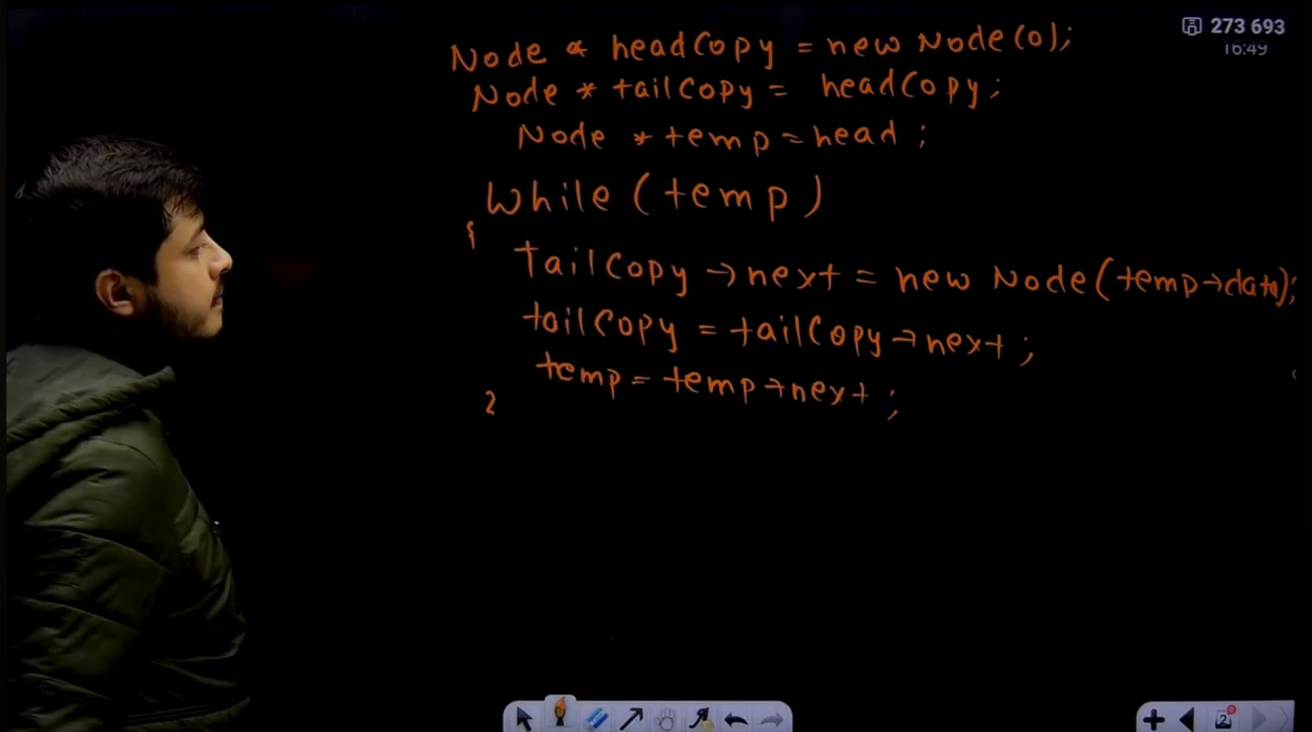


Clone a Linked List

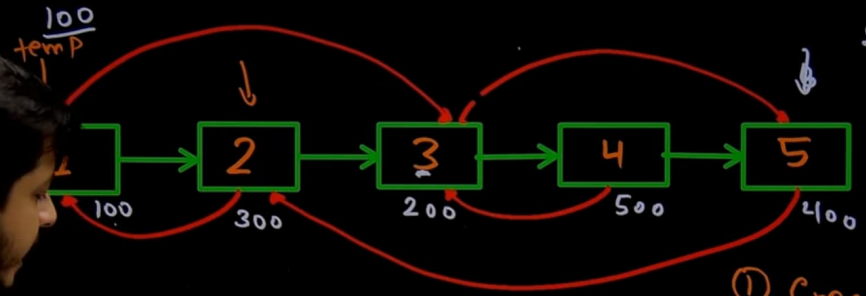


Clone a Linked List




Node * headCopy = new Node(0);
Node * tailCopy = headCopy;
Node * temp = head;
While (temp)
{
tailCopy → next = new Node(temp → data);
tailCopy = tailCopy → next;
temp = temp → next;
}

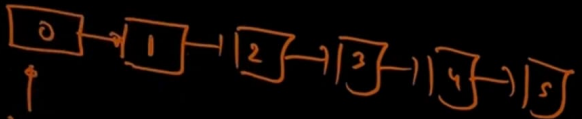
Clone a Linked List

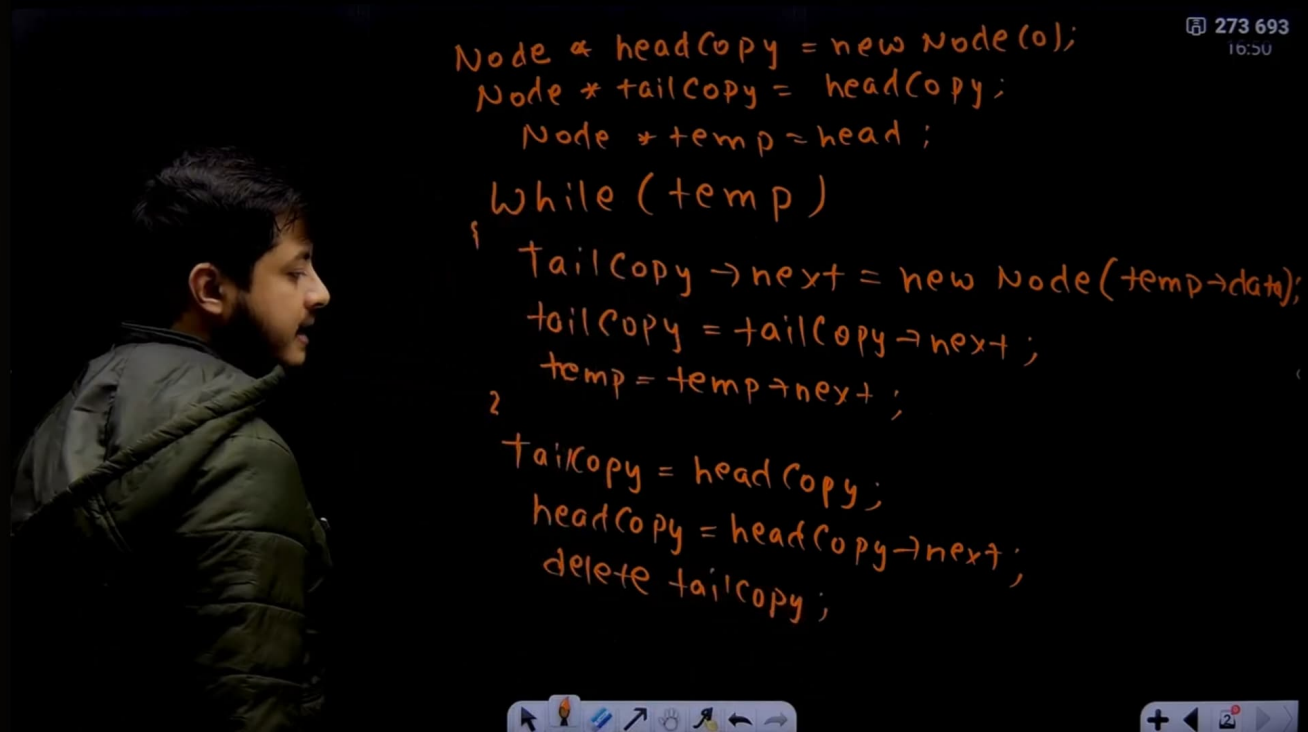



```
Class Node
{ public:
  int data;
  Node *next,
       *arb
};
```

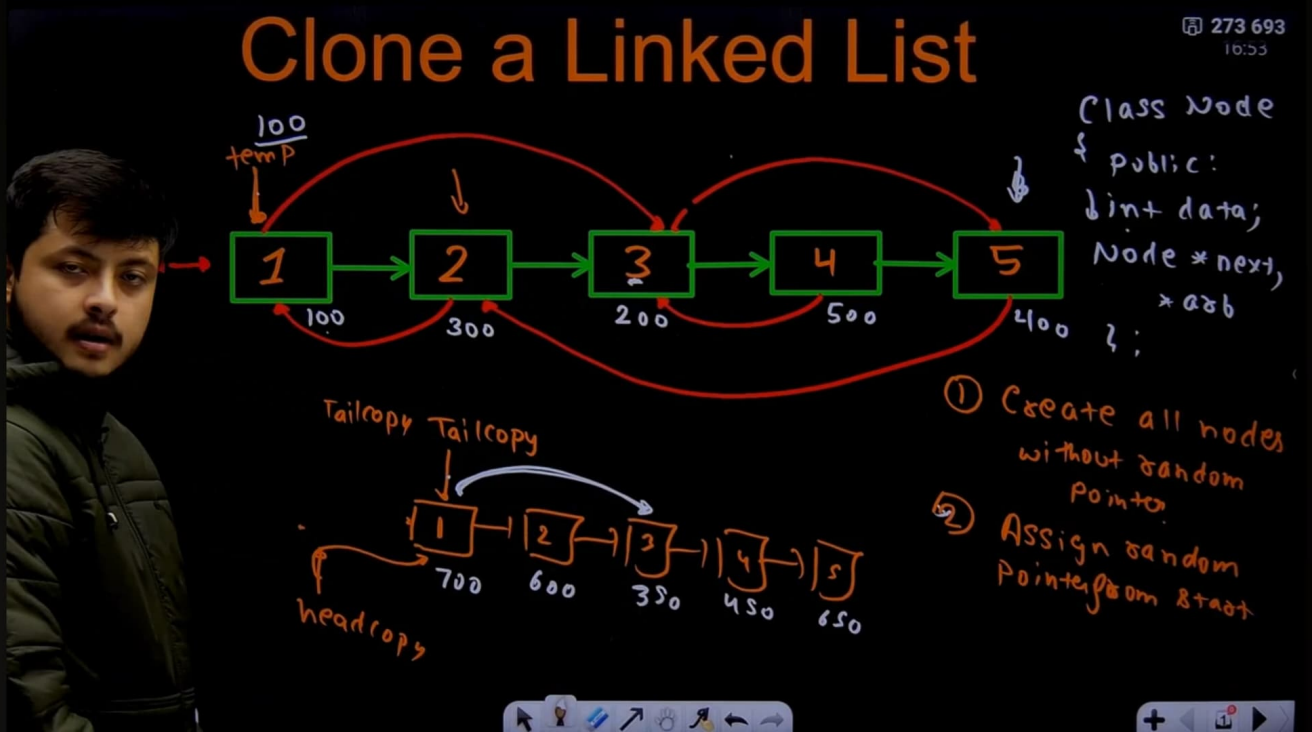
① Create all nodes without random pointer.

② Assign random pointer from start




Node * headCopy = new Node(0);
Node * tailCopy = headCopy;
Node * temp = head;
While (temp)
{
tailCopy → next = new Node(temp → data);
tailCopy = tailCopy → next;
temp = temp → next;
}
tailCopy = headCopy;
headCopy = headCopy → next;
delete tailCopy;

Clone a Linked List

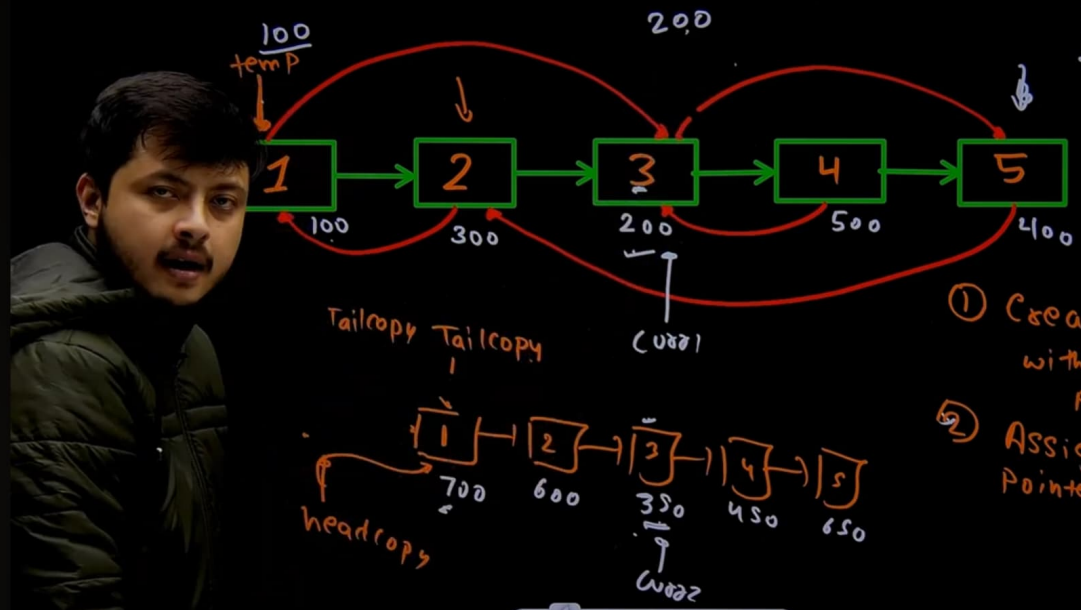



```
Class Node
{ public:
  int data;
  Node *next,
       *arb
};
```

① Create all nodes without random pointer.

② Assign random pointer from start

Clone a Linked List



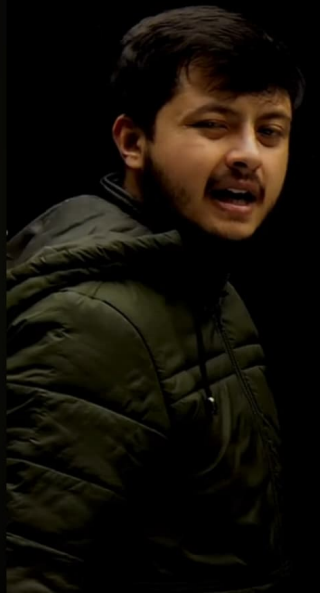

```
Class Node
{ public:
int data;
Node *next,
     *arb
};
```

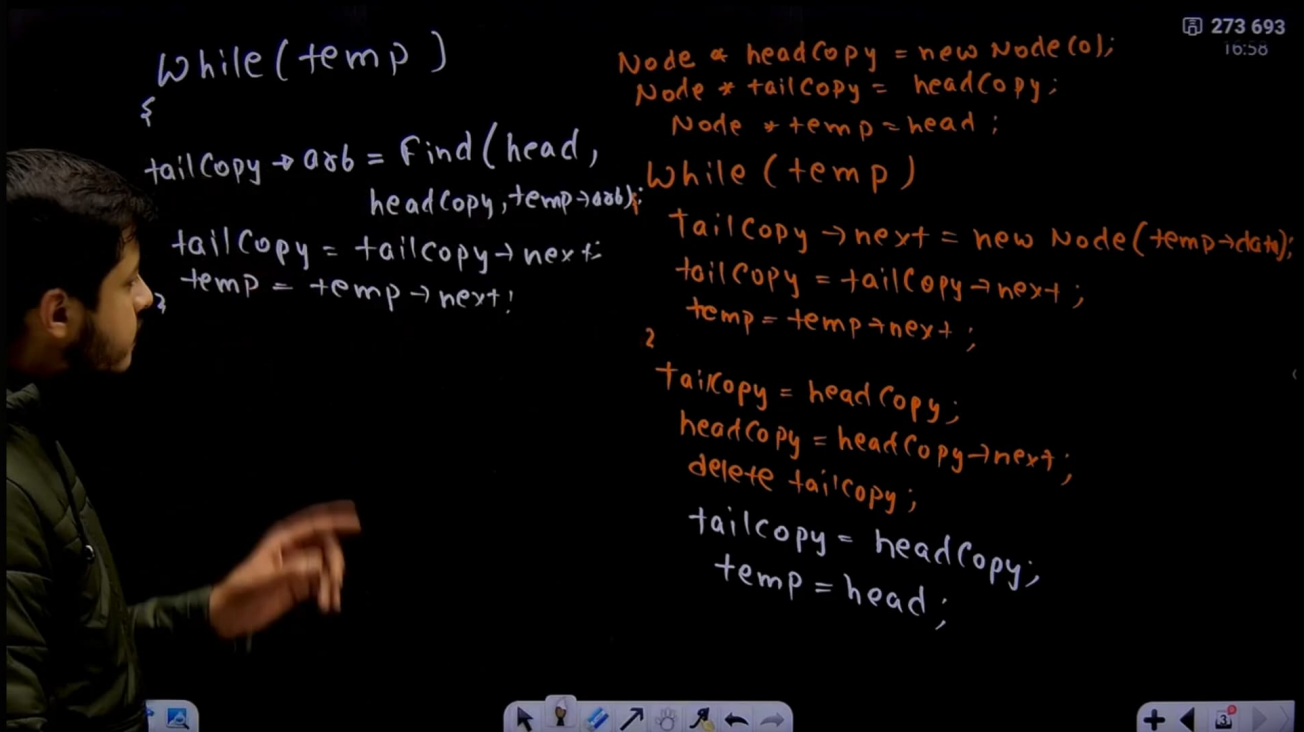
① Create all nodes without random pointer.

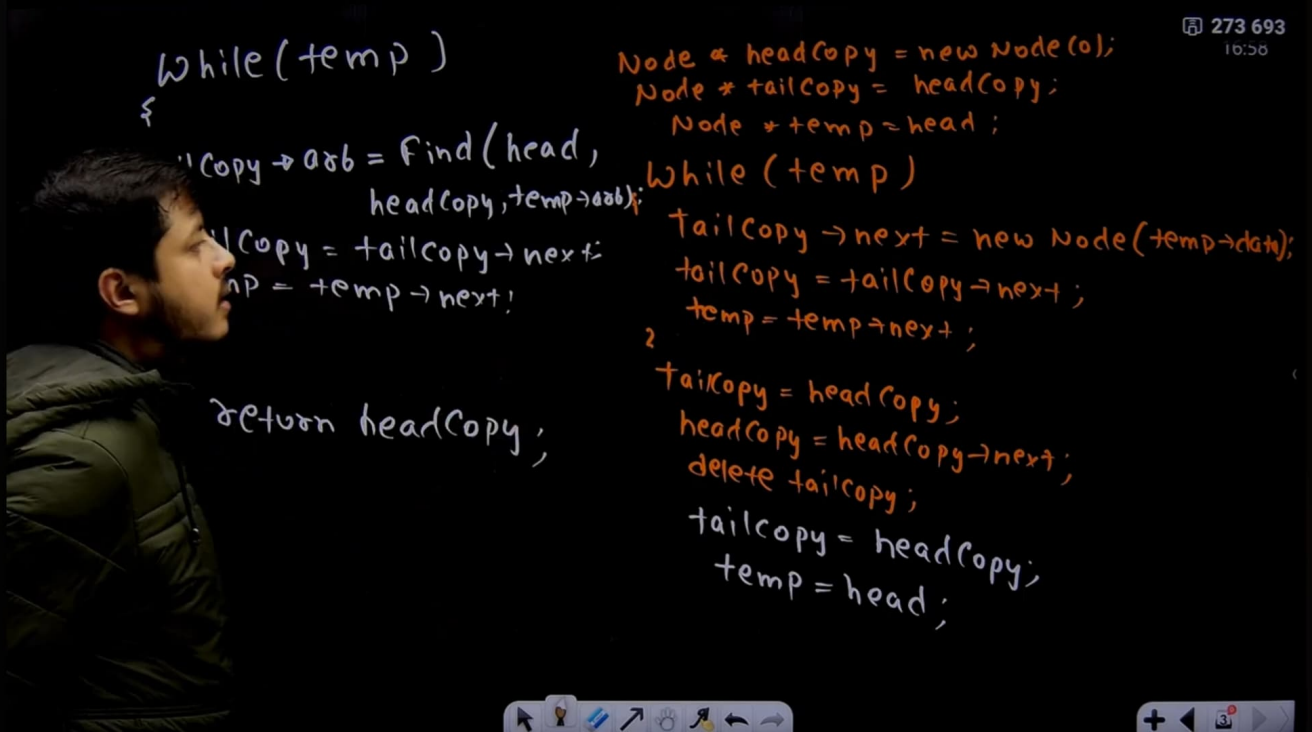
② Assign random pointer from start

```
Node * (Node *curr1, Node *curr2,
                                    Node * x)
{

    while(curr1 != x)
    {
        curr1 = curr1->next;
        curr2 = curr2->next;
    }

    return curr2;

}
```

```
Node *  ( Node *curr1 , Node *curr2 ,
{       if ( x == NULL)                Node * x)
            return NULL;

        while (curr1 != x )
        {
            curr1 = curr1 -> next;
            curr2 = curr2 -> next;
        }

        return curr2;

}
```

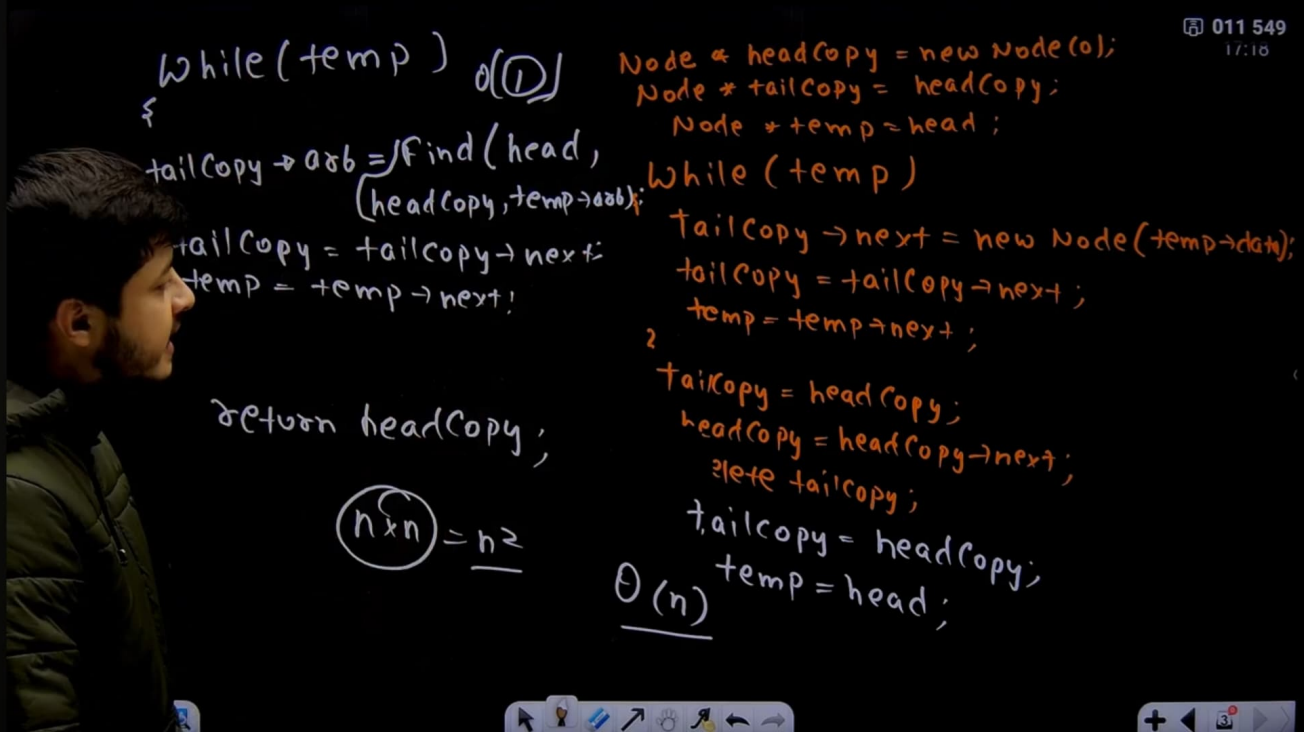



While (temp)
{
tailCopy → arb = Find (head, headCopy, temp → arb);
tailCopy = tailCopy → next;
temp = temp → next;
}
Node * headCopy = new Node(0);
Node * tailCopy = headCopy;
Node * temp = head;
While (temp)
tailCopy → next = new Node(temp → data);
tailCopy = tailCopy → next;
temp = temp → next;
tailCopy = headCopy;
headCopy = headCopy → next;
delete tailCopy;
tailcopy = headCopy;
temp = head;
273 693
10:58

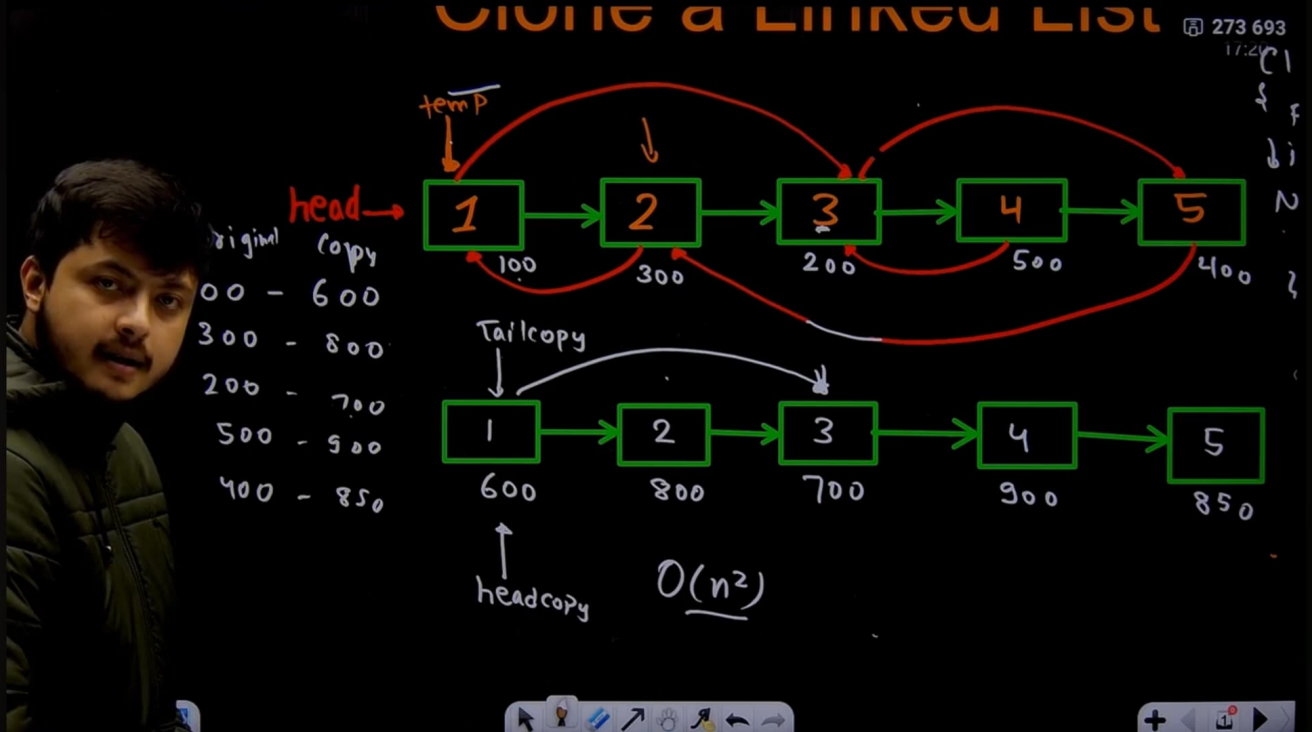


```
while(temp)
{
    'lcopy->arb = Find(head,
                       headcopy, temp->arb);
    'lcopy = tailcopy->next;
    np = temp->next;

return headcopy;
```

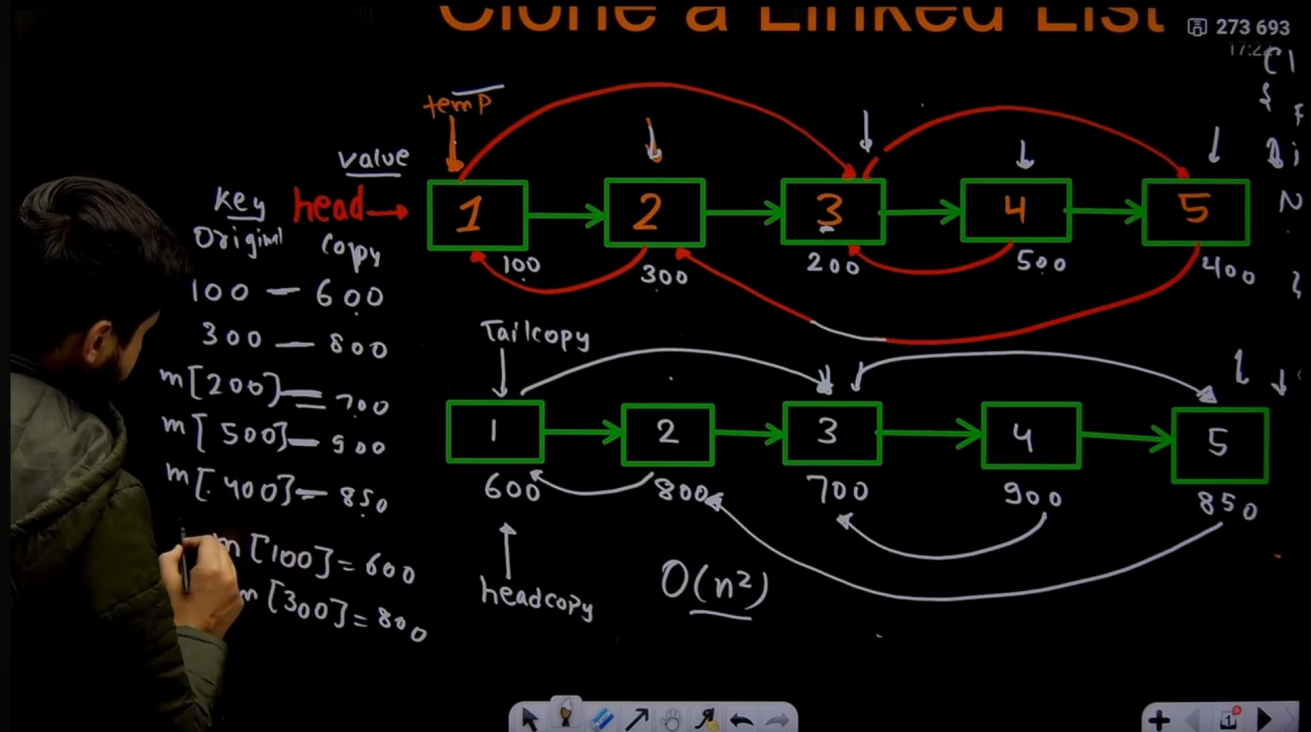
```
Node * headcopy = new Node(0);
Node * tailcopy = headcopy;
  Node * temp = head;
while(temp)
{
  tailcopy->next = new Node(temp->data);
  tailcopy = tailcopy->next;
  temp = temp->next;
}
tailcopy = headcopy;
 headcopy = headcopy->next;
 delete tailcopy;
 tailcopy = headcopy;
  temp = head;
```


While (temp) O(1)
{
tailCopy → arb = Find (head, headCopy, temp → arb);
tailCopy = tailCopy → next;
temp = temp → next;
return headCopy;
n × n = n²
Node * headCopy = new Node(0);
Node * tailCopy = headCopy;
Node * temp = head;
While (temp)
tailCopy → next = new Node(temp → data);
tailCopy = tailCopy → next;
temp = temp → next;
2
tailCopy = headCopy;
headCopy = headCopy → next;
delete tailCopy;
tailCopy = headCopy;
temp = head;
O(n)
011 549
17:18

Clone a Linked List

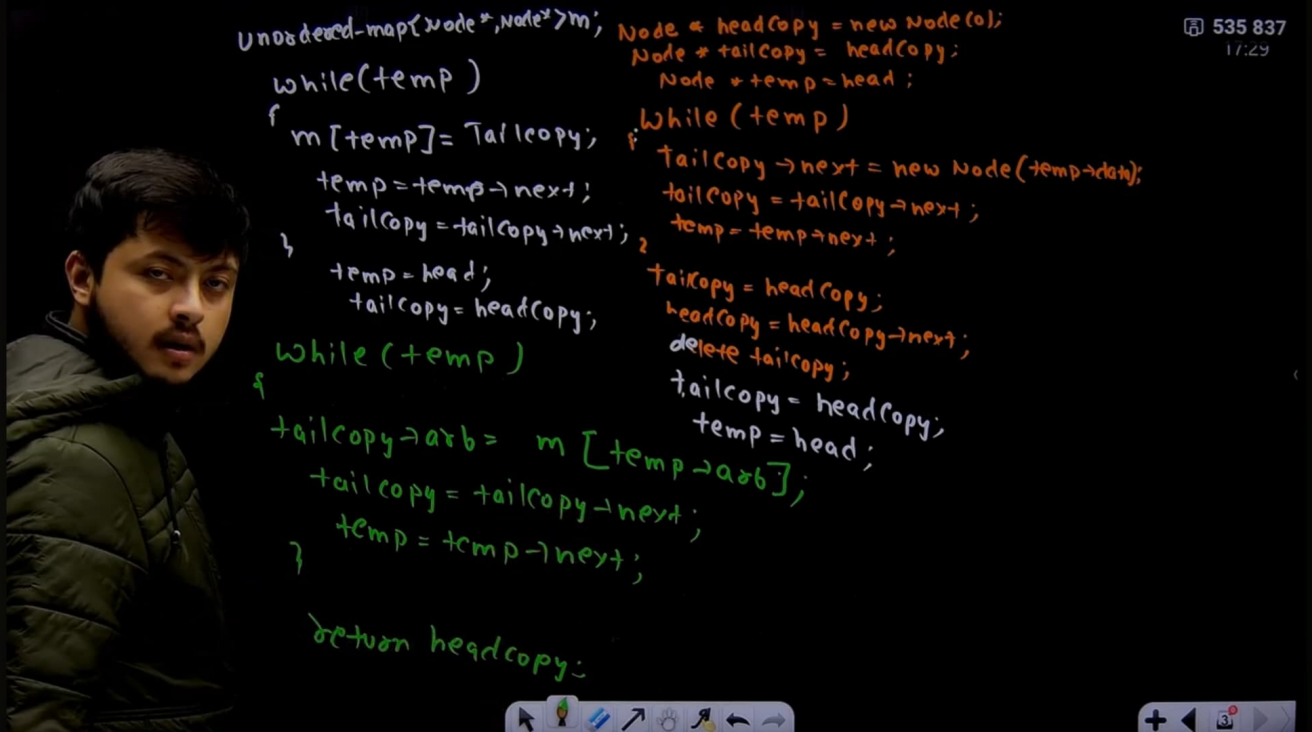


Clone a Linked List



```
unordered_map<Node*, Node*> m;

while(temp)
{
    m[temp] = Tailcopy;
    temp = temp->next;
    tailcopy = tailcopy->next;
}
```

```
Node * headcopy = new Node(0);
Node * tailcopy = headcopy;
Node * temp = head;
While (temp)
{
    tailcopy->next = new Node(temp->data);
    tailcopy = tailcopy->next;
    temp = temp->next;
}
tailcopy = headcopy;
headcopy = headcopy->next;
delete tailcopy;
tailcopy = headcopy;
temp = head;
```


Unordered_map<Node*,Node*>m;
while(temp)
{
m[temp]= Tailcopy;
temp=temp->next;
tailcopy=tailcopy->next;
}
temp=head;
tailcopy=headcopy;
while(temp)
{
tailcopy->arb= m[temp->arb];
tailcopy=tailcopy->next;
temp=temp->next;
}
return headcopy;
Node * headcopy = new Node(0);
Node * tailcopy = headcopy;
Node * temp=head;
while(temp)
{
tailcopy->next = new Node(temp->data);
tailcopy=tailcopy->next;
temp=temp->next;
}
tailcopy=headcopy;
headcopy=headcopy->next;
delete tailcopy;
tailcopy=headcopy;
temp=head;
535 837
17:29

Problem | Editorial | Submissions | Comments

Clone a linked list with next and random pointer

Hard Accuracy: 64.8% Submissions: 66K+ Points: 8

You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** i.e. **a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y**.

Return the head of the copied linked list.

C++ (g++ 5.4) Start Timer

```
31      Node *headCopy = new Node(0);
32      Node *tailCopy = headCopy;
33      Node *temp = head;
34
35      // Clone created without random pointer
36      while(temp)
37      {
38          tailCopy->next = new Node(temp->data);
39          tailCopy = tailCopy->next;
40          temp=temp->next;
41      };
42
43      tailCopy = headCopy;
44      headCopy = headCopy->next;
45      delete tailCopy;
46
47      tailCopy = headCopy;
48      temp = head;
49
50
51
52
```

Problem Editorial Submissions Comments

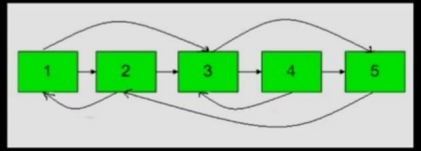
Today!

You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** i.e. **a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y**.

Return the head of the copied linked list.



C++ (g++ 5.4) Start Timer

```
43      tailCopy = headCopy;
44      headCopy = headCopy->next;
45      delete tailCopy;
46
47      tailCopy = headCopy;
48      temp = head;
49
50      unordered_map<Node*,Node*>m;
51      // fill the value inside map
52      while(temp)
53      {
54          m[temp] = tailCopy;
55          tailCopy = tailCopy->next;
56          temp=temp->next;
57      }
58
59
60      // Assign randome pointer to Clone LL
61
62
63
```

Problem Editorial Submissions Comments

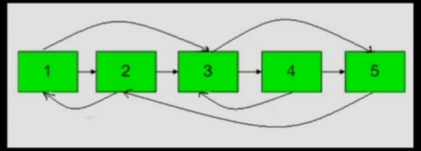
Today!

You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** i.e. **a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y**.

Return the head of the copied linked list.



C++ (g++ 5.4) Start Timer

```
57      }
58
59
60      // Assign randome pointer to Clone LL
61
62      tailCopy = headCopy;
63      temp = head;
64
65      while(temp)
66      {
67          tailCopy->arb = m[temp->arb];
68          tailCopy = tailCopy->next;
69          temp=temp->next;
70      }
71
72
73      return headCopy;
74
75
76
77
```